

Single Agent Systems & Agent Pipelines - Quiz

Q1. Explain the concept of a stateful directed graph in agent pipelines. How does it differ from a simple linear pipeline?

A stateful directed graph is an agent workflow where each node performs a specific task, and edges define the flow of execution. The graph maintains a shared state (memory) that stores information, allowing different nodes to read or update it during execution.

A linear pipeline executes tasks in a fixed sequence (Step 1 → Step 2 → Step 3) without changing the path. In contrast, a stateful directed graph supports branching, looping, and conditional routing based on the current state.

Difference:

- Linear pipeline: Fixed execution order.
- Stateful directed graph: Dynamic execution with shared memory and multiple possible paths.

Q2. Describe the role of nodes and edges in an agent workflow. Give an example of each.

In an agent workflow:

- Nodes represent individual operations or tasks performed by the agent.
- Edges define how control moves from one node to another.

Example of a node: A node that summarizes a user's document.

Example of an edge: After document summarization, an edge directs the workflow to a translation node if the user requests another language. Thus, nodes perform actions, while edges control the execution flow.

Q3. What is conditional routing in an agent system? Design a simple rule-based routing logic for three different query types.

Conditional routing is the process of selecting different execution paths based on predefined conditions or user input.

Rule-based routing example:

- If the query is mathematical, send it to the Calculator Tool.
- If the query is coding-related, send it to the Code Generation Tool.
- If the query is general knowledge, send it to the Knowledge Retrieval Tool.

This routing ensures that each query is handled by the most suitable tool.

Q4. Why are cycles (loops) important in agent pipelines? Provide a use case where a retry loop is necessary.

Cycles (loops) allow an agent to repeat a task until a desired condition is met. They improve reliability by enabling retries and corrections.

Use case: If an API request fails due to a temporary network issue, the agent retries the request up to three times before reporting an error. This retry loop increases the chances of successful execution.

Q5. Explain how a single-agent system can simulate multi-agent behavior internally.

A single-agent system can simulate multiple agents by dividing a complex task into different internal roles. The same agent sequentially performs these roles, such as planning, reasoning, executing, and reviewing.

For example, the agent first creates a plan, then generates a solution, and finally checks its own output for errors. Although only one agent exists, it behaves like multiple specialized agents working together.

Q6. What are JSON schema tools? How do they help in structuring tool inputs and outputs?

JSON Schema tools define the expected format, data types, and required fields for tool inputs and outputs.

They help by:

- Validating input data before execution.
- Ensuring consistent output structure.
- Reducing errors caused by missing or incorrect fields.
- Making communication between the agent and tools more reliable.

For example, a weather tool may require:

```
{  
  "city": "Delhi",  
  "unit": "Celsius"  
}
```

Q7. Compare sequential tool calls and parallel tool calls. When would you prefer one over the other?

Sequential Tool Calls	Parallel Tool Calls
Tools execute one after another.	Multiple tools execute simultaneously.
Output of one tool is used by the next.	Tools work independently.
Slower but suitable for dependent tasks.	Faster for independent tasks.

Preference:

- Use sequential calls when later tasks depend on earlier results.
- Use parallel calls when tasks are independent and need faster execution.

Q8. How would you implement error handling in a tool-using agent? Provide at least two strategies.

Error handling improves the reliability of an agent.

Two common strategies are:

1. Retry Mechanism: If a tool fails due to a temporary issue, the agent retries the operation a limited number of times.
2. Fallback Strategy: If the primary tool fails, the agent switches to an alternative tool or provides a default response.

Additional strategies include validating inputs before execution and logging errors for debugging.

Q9. What is trajectory evaluation in agent systems? Why is it important beyond just checking final output?

Trajectory evaluation is the process of assessing the entire sequence of actions and decisions made by an agent while completing a task.

It is important because it:

- Identifies mistakes in reasoning steps.
- Detects unnecessary tool calls.
- Measures efficiency and correctness throughout the workflow.
- Helps improve future agent performance.

Thus, it evaluates both how the answer was produced and what the final answer is.

Q10. Define task completion rate and cost metrics. How would you measure and optimize them in a real-world system?

Task Completion Rate (TCR) is the percentage of tasks successfully completed by the agent.

Formula:

$$\text{Task Completion Rate} = \frac{\text{Successful Tasks}}{\text{Total Tasks}} \times 100$$

Cost metrics measure the resources consumed, such as API cost, execution time, token usage, or computational resources.

Measurement:

- Track the number of successful tasks.
- Record API calls, execution time, and token usage.

Optimization:

- Reduce unnecessary tool calls.
- Use parallel execution for independent tasks.
- Improve routing decisions.
- Cache frequently used results.
- Optimize prompts to reduce token usage.

These improvements increase task success while lowering overall operational cost.